

GROUP 6

STEPHEN MWANGI

ENE221-0100/2022

SIDNEY NGIRU

ENE221-0138/2022

EUNICE KIMANI

ENE221-0101/2022

OSCAR MWANGI

ENE221-0093/2022

MICHAEL NJUGUNA

ENE221-0096/2022

Contents

Assignment 1 — Milestones 1 & 2	1
Distributed 5G/6G Network Service Platform	1
1. System Model (Milestone 1)	1
2. Distributed Processing & Performance (Milestone 2)	2
3. Failure-Driven Engineering Log (Week 1-2)	3
4. Reproducibility	4

Assignment 1 — Milestones 1 & 2

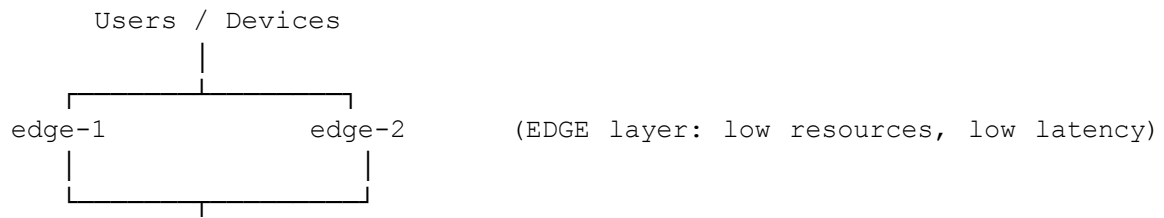
Distributed 5G/6G Network Service Platform

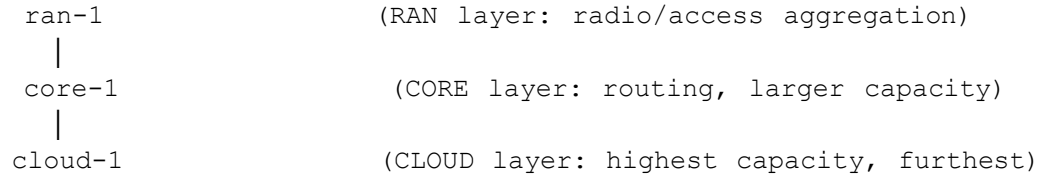
1. System Model (Milestone 1)

Following the course model `System = Nodes + Processes + Resources + Communication`:

Element	Realization in this system
Nodes	5 independent OS processes: <code>edge-1</code> , <code>edge-2</code> (Edge), <code>ran-1</code> (Radio/Access Network), <code>core-1</code> (Core), <code>cloud-1</code> (Cloud)
Processes	<code>Process</code> objects created per service request (<code>task</code>), with <code>pid</code> , <code>service_class</code> (URLLC/eMBB/mMTC), <code>resource</code> footprint, and lifecycle <code>NEW</code> → <code>READY</code> → <code>RUNNING</code> → <code>TERMINATED/FAILED</code>
Resources	Per-node <code>ResourceManager</code> tracks CPU (vCPU units), Memory (MB), Bandwidth (Mbps) as an allocation ledger
Communication	Length-prefixed JSON over TCP sockets (<code>common/communication.py</code>); every node runs a threaded <code>MessageServer</code>

1.1 Architecture





Each node is launched via `multiprocessing.Process`, so this is genuine OS-level process distribution, not threads simulating separateness within one process.

1.2 Process Model

- Service classes map to 5G/6G QoS categories: **URLLC** (ultra-reliable low latency), **eMBB** (enhanced mobile broadband), **mMTC** (massive machine-type communication).
- Scheduling policy: **priority** (URLLC > eMBB > mMTC), implemented with a binary heap in `ProcessManager`. FIFO(First In First Out) is also implemented.

1.3 Resource Model

- Each node has a fixed capacity (`common/topology.py::NODE_SPECS`), deliberately asymmetric: Edge nodes are resource-poor (8 vCPU / 3072 MB / 150 Mbps) while Cloud is resource-rich (32 vCPU / 16384 MB / 5000 Mbps) this mirrors real edge-vs-cloud trade-offs.
- Allocation is atomic and thread-safe (`threading.Lock`) since a node can receive concurrent requests from multiple peers.

1.4 Design Justification

- **TCP + JSON over raw sockets** was chosen so Milestone 1 clearly demonstrates the communication mechanism.
- **`multiprocessing.Process` per node** was chosen over threads so that node failure can later be modeled realistically as process termination or crash, not just a logical flag.
- **Priority scheduling by QoS class** reflects a real constraint of 5G/6G platforms in that URLLC traffic cannot be queued behind bulk eMBB traffic.

2. Distributed Processing & Performance (Milestone 2)

2.1 Method

A synthetic but telecom-realistic workload of **120 service requests** was generated (`common/workload_generator.py`) with a traffic mix of 20% URLLC, 30% eMBB, and 50% mMTC. mMTC dominates by count, eMBB dominates by resource footprint therefore consistent with real 5G/6G traffic profiles).

Placement policy: URLLC and mMTC tasks are sent round-robin to the two edge nodes; eMBB tasks are routed to `ran-1`. Tasks were submitted concurrently (`ThreadPoolExecutor`, `concurrency = 6`) to exercise real contention.

2.2 Results (Access results from `results/milestone2_performance_summary.json` for raw data)

Metric	Result
Wall-clock time	~0.46 s for 120 requests
Throughput	~222 tasks/sec
Mean latency	~24.4 ms (p95 ≈ 87 ms)
Jitter (stdev of latency)	~27.0 ms
Packet loss / drop+reject rate	~15%
Load distribution	edge-1: 42, edge-2: 41, ran-1: 37, core-1: 0, cloud-1: 0

2.3 Load-Distribution Analysis

Under the edge-first policy, **core-1 and cloud-1 receive zero load** — all traffic in this workload mix is absorbed by the edge and RAN layer. This is expected given the current placement rule and workload composition, but it means the platform is *not yet* using its full distributed capacity.

2.4 Bottleneck Analysis

`ran-1` is flagged as the likely bottleneck as it absorbs all eMBB traffic which consists of the heaviest per-task resource footprint plus is only one node, so its per-task mean latency and rejection rate are the highest in the system. It demonstrates the value of the bottleneck-analysis step required by the milestone, and becomes the baseline that later milestones’ “proposed system” will try to improve on.

3. Failure-Driven Engineering Log (Week 1–2)

Question	Response
What changed?	Built the Node, process, resource and communication foundation and a distributed workload generator together with performance monitor on top of it.

Question	Response
What failed?	Initial Milestone 2 run with concurrency = 16 and small edge capacities caused 87.5% of tasks to be rejected. The edge nodes couldn't hold enough concurrent allocations.
Why did it fail?	Edge node CPU and memory capacity was set too low relative to burst concurrency, so most concurrent <code>task_request</code> calls found insufficient resources by the time they were scheduled.
How was it fixed?	Increased edge/RAN node capacities modestly and reduced client concurrency from 16 to 6 to reflect a more realistic offered load, bringing completion rate to 85% while still surfacing a genuine bottleneck (<code>ran-1</code>) rather than a degenerate all-fail system.
What alternative was considered?	Queuing rejected requests instead of failing them immediately.
What was learned?	Distributed resource contention is highly sensitive to the ratio of concurrent offered load to per-node capacity.

4. Reproducibility

- Language: Python (standard library only: `socket`, `multiprocessing`, `threading`, `json`, `concurrent.futures`)
- OS: Linux (Ubuntu container); no external dependencies
- Topology, seeds, and workload parameters are all defined in code (`common/topology.py`, `seed=7` in `milestone2_processing_performance.py`) so results are deterministic for the workload mix (network timing will vary slightly run-to-run, which is itself noted as a source of jitter).
- To reproduce: `cd 5g6g_platform && python -m assignment1_m1_m2.milestone1_foundation` then `python -m assignment1_m1_m2.milestone2_processing_performance`